

ResumeFlow AI

Product Requirements Document — MVP Specification

Version 1.0 | June 27, 2026 | Confidential

PRODUCT	ResumeFlow AI
TYPE	AI-Powered Job Search Platform (SaaS)
STACK	React 19 · TypeScript · Express · MongoDB · OpenAI/OpenRouter · Cloudinary
AUTHOR	Ahmed
STATUS	In Development — MVP Phase
DATE	June 27, 2026

Executive Summary

ResumeFlow AI transforms the fragmented job search experience into a single, intelligent platform. Job seekers today juggle separate tools for resume building, AI tailoring, application tracking, and recruiter engagement — ResumeFlow AI unifies all four. By combining an AI-powered resume engine, a Kanban application tracker, a cover letter generator, and real-time recruiter analytics, the platform creates a sticky daily-use product that competes directly with Teal, Enhancv, and Kickresume — while outperforming each of them individually.

Strategic Positioning

Most competitors solve one part of the job search problem well. ResumeFlow AI is the only platform that combines Resume Builder + AI Optimizer + Job Tracker + Recruiter Analytics in a single cohesive workflow.

Core User Journey

The platform is designed around a single, end-to-end job search workflow. Every feature reinforces the next, driving daily retention.

1	Register / Login	Auth.js — credentials + OAuth (Google, GitHub)
2	Dashboard	Central hub — metrics at a glance
3	Upload CV	PDF or DOCX, stored via Cloudinary
4	AI Parse CV	Extract structured data from raw CV
5	Build Master Resume	Editable, versioned source-of-truth resume
6	Analyze Job Description	AI extracts required skills & keywords
7	Generate Resume Version	Tailored resume + ATS match score
8	Generate Cover Letter	Company & role-specific, AI-written
9	Apply & Track	Log application, set status & deadlines
10	Recruiter Analytics	See when & how often your resume is viewed

MVP Feature Breakdown

Authentication

- Auth.js (formerly NextAuth.js) — framework-agnostic auth library for Express + React
- Supports: Email/password credentials provider + OAuth (Google, GitHub) out of the box
- Session management via database-backed sessions (MongoDB adapter) or JWT strategy
- Protected route middleware using Auth.js session validation on every API request
- User profile with avatar — pulled from OAuth provider or uploaded via Cloudinary

CV Upload & AI Parsing

- Upload PDF or DOCX — stored on Cloudinary
- AI extracts: contact info, summary, experience, skills, education, projects
- Structured data mapped to MongoDB resume schema
- Handles varied CV formats and layouts gracefully

Master Resume Builder

- Fully editable sections: summary, experience, skills, education, projects, achievements
- Rich inline editing with section reordering
- Resume versioning — every tailored version saved separately
- Single master resume as the source of truth for all versions

AI Resume Tailoring

- Paste job description → AI analyzes required skills & keywords
- AI generates a tailored resume version optimized for the role
- ATS match score shown (0–100%) with specific improvement suggestions
- Multiple templates available for different industries

PDF Export

- Export any resume version as a polished, ATS-friendly PDF
- Template-aware rendering — layout matches selected resume template
- Downloadable and shareable via unique public tracking link

Cover Letter Generator

- AI generates a personalized cover letter per job application
- Inputs: resume version + job description + company name
- Editable before saving or sending
- Stored and linked to the corresponding application

Application Tracker

- Kanban board view: Saved → Applied → Screening → Interview → Technical Assessment → Offer / Rejected
- Table view for sorting and filtering applications
- Each application links to: company, role, resume version, cover letter, status, notes, deadlines
- Visual pipeline overview on the dashboard

Recruiter Analytics

- Unique shareable tracking link per resume version (e.g., [resumeflow.ai/r/ahmed-google-fe](#))
- Events tracked: opens, downloads, unique visitors, timestamps
- Dashboard shows: 'Google recruiter viewed resume 3 times · Last seen 2 days ago'
- IP & user-agent captured per event for deduplication
- Premium differentiator — most competitors lack this entirely

Application Pages

Public Pages

ROUTE	PAGE	DESCRIPTION
/	Landing Page	Hero, Features, Resume Examples, CTA — conversion-focused public entry point
/login	Login	Auth.js sign-in page — credentials or OAuth (Google, GitHub)
/register	Register	Auth.js sign-up with email/password or one-click OAuth

Protected Pages (Authenticated)

ROUTE	PAGE	DESCRIPTION
/dashboard	Dashboard	Widgets: total resumes, versions, applications, interviews, offers, recruiter views
/resume/upload	CV Upload	Drag-and-drop PDF/DOCX upload with AI parsing trigger
/resume/:id	Master Resume Builder	Full resume editor with section management and version history
/resume/:id/generate	Generate Resume Version	Job description input → AI tailoring → match score → generated version
/resume/version/:id	Resume Version Detail	View, edit, export PDF, generate cover letter, create tracking link
/resume/version/:id/analytics	Recruiter Analytics	Total opens, downloads, unique viewers, event timeline
/applications	Application Tracker	Kanban and table view of all job applications
/applications/:id	Application Detail	Company, role, status, linked resume version, cover letter, notes, deadlines
/cover-letters	Cover Letters	List of all generated cover letters with quick access
/settings	Settings	Profile preferences, account management, notification settings

REST API Endpoints

Authentication

POST	/api/auth/register	Auth.js credentials signup — hashes password, creates user
POST	/api/auth/login	Auth.js credentials sign-in — returns session token
POST	/api/auth/logout	Destroy Auth.js session
GET	/api/auth/session	Return current Auth.js session (user + token)

GET	/api/auth/callback/:provider	OAuth callback handler (Google, GitHub)
-----	------------------------------	---

Resumes

POST	/api/resumes	Create a new master resume
GET	/api/resumes	List all resumes for current user
GET	/api/resumes/:id	Get a single resume by ID
PUT	/api/resumes/:id	Update resume content
DELETE	/api/resumes/:id	Delete resume and all versions

Resume Versions

POST	/api/versions	Create a new tailored resume version
GET	/api/versions/:id	Get a specific version
PUT	/api/versions/:id	Update a version
GET	/api/resumes/:id/versions	List all versions for a resume

AI Services

POST	/api/ai/parse-cv	Extract structured data from uploaded CV
POST	/api/ai/analyze-job	Analyze job description for keywords & requirements
POST	/api/ai/generate-version	Generate tailored resume with match score
POST	/api/ai/improve-section	Suggest improvements for a specific section
POST	/api/ai/generate-cover-letter	Generate personalized cover letter

Applications

POST	/api/applications	Create new application record
GET	/api/applications	List all applications
GET	/api/applications/:id	Get single application detail
PUT	/api/applications/:id	Update status, notes, deadlines
DELETE	/api/applications/:id	Remove application

Analytics

GET	/api/analytics/:resumeVersionId	Aggregate analytics for a version
POST	/api/analytics/track-view	Log recruiter view event

POST	/api/analytics/track-download	Log recruiter download event
------	-------------------------------	------------------------------

Export

POST	/api/export/pdf	Generate and return PDF for a resume version
------	-----------------	--

Database Schema — MongoDB Collections

users

FIELD	TYPE	NOTES
<code>_id</code>	ObjectId	Auto-generated primary key
<code>name</code>	String	Full display name
<code>email</code>	String	Unique, indexed, lowercase
<code>passwordHash</code>	String	Auth.js hashed password — only set for credentials provider
<code>emailVerified</code>	Date	Set by Auth.js on email verification — null until verified
<code>provider</code>	String	Auth provider: 'credentials' 'google' 'github'
<code>avatar</code>	String	Cloudinary URL
<code>createdAt</code>	Date	Account creation timestamp

resumes

FIELD	TYPE	NOTES
<code>_id</code>	ObjectId	Primary key
<code>userId</code>	ObjectId	Ref → users
<code>title</code>	String	User-defined resume label
<code>contact</code>	Object	Name, email, phone, location, LinkedIn, GitHub
<code>summary</code>	String	Professional summary paragraph
<code>experiences</code>	Array	Each: title, company, dates, bullets[]
<code>projects</code>	Array	Each: name, description, techStack[], url
<code>skills</code>	Array	Grouped skill categories
<code>education</code>	Array	Degree, institution, dates, GPA
<code>achievements</code>	Array	Awards, certifications, publications
<code>createdAt</code>	Date	Creation timestamp

resume_versions

FIELD	TYPE	NOTES
<code>_id</code>	ObjectId	Primary key
<code>userId</code>	ObjectId	Ref → users
<code>resumeId</code>	ObjectId	Ref → resumes (parent)
<code>versionName</code>	String	e.g., 'Google SWE — June 2025'
<code>companyName</code>	String	Target company name
<code>jobTitle</code>	String	Target job title

jobDescription	String	Raw job description text
template	String	Selected template identifier
matchScore	Number	ATS match score (0–100)
generatedResume	Object	AI-tailored resume content
publicLinkSlug	String	Unique slug for tracking link — indexed
createdAt	Date	Generation timestamp

applications

FIELD	TYPE	NOTES
_id	ObjectId	Primary key
userId	ObjectId	Ref → users
companyName	String	Company applied to
jobTitle	String	Role applied for
resumeVersionId	ObjectId	Ref → resume_versions
coverLetterId	ObjectId	Ref → cover_letters (nullable)
status	String	Enum: Saved Applied Screening Interview Technical Offer Rejected
appliedDate	Date	When application was submitted
notes	String	Free-text notes
deadlines	Array	Interview dates, task deadlines
createdAt	Date	Record creation timestamp

cover_letters

FIELD	TYPE	NOTES
_id	ObjectId	Primary key
userId	ObjectId	Ref → users
resumeVersionId	ObjectId	Ref → resume_versions
companyName	String	Target company
jobTitle	String	Target role
content	String	Full cover letter text (editable)
createdAt	Date	Generation timestamp

recruiter_analytics

FIELD	TYPE	NOTES
_id	ObjectId	Primary key
resumeVersionId	ObjectId	Ref → resume_versions — indexed
eventType	String	Enum: view download

ipAddress	String	Hashed for privacy
userAgent	String	Browser/device fingerprint
createdAt	Date	Event timestamp

Project Structure — Monorepo

A monorepo structure is strongly recommended. It keeps the client, server, and any shared types/utilities in a single repository, simplifying deployment, type sharing, and CI/CD pipelines.

client/src/	server/src/
components/ pages/ hooks/ services/ store/ layouts/ lib/ types/	controllers/ routes/ services/ middlewares/ models/ utils/ config/ prompts/

Tech Stack

Frontend

- React 19 — latest concurrent rendering features
- TypeScript — full type safety across client and server
- Tailwind CSS v4 — utility-first with CSS variable theming
- shadcn/ui — accessible, unstyled component primitives
- Light / Dark / System theme support

Backend

- Express.js (TypeScript) — clean MVC-style REST API
- Mongoose — typed MongoDB ODM with schema validation
- Auth.js — credentials + OAuth providers, MongoDB session adapter
- Multer + Cloudinary — file uploads with cloud CDN delivery

AI Layer

- OpenRouter or OpenAI — switchable via environment config
- Dedicated prompt files per AI action (parse, tailor, cover letter, analyze)
- Structured JSON output enforced for all AI responses
- Token usage tracked per user for future rate limiting / billing

DevOps & Infrastructure

- MongoDB Atlas — managed cloud database with automated backups
- Cloudinary — resume file and avatar storage CDN
- Environment-based config (.env) for all secrets
- Recommended deploy: Railway / Render (API) + Vercel (client)

Development Roadmap — MVP Phases

Phase 1 — Foundation

- ✓ Project scaffolding: monorepo, ESLint, Prettier, TypeScript config
- ✓ Express server setup with middleware (CORS, helmet, morgan, error handler)
- ✓ MongoDB connection, Mongoose models for users and resumes
- ✓ Auth.js setup: credentials provider + Google/GitHub OAuth, MongoDB adapter for sessions
- ✓ CV upload endpoint with Cloudinary integration

Phase 2 — Resume Core

- AI CV parsing service (OpenAI/OpenRouter) with structured JSON output
- Master resume builder API (CRUD) with Mongoose schema
- Resume versioning: generate tailored version from job description
- ATS match scoring returned alongside generated version
- PDF export endpoint using headless rendering

Phase 3 — Job Search Tools

- Cover letter generator API (linked to resume version + job description)
- Application tracker CRUD with status state machine
- Kanban and table view data endpoints
- Application linked to resume version and cover letter

Phase 4 — Analytics & Dashboard

- Recruiter tracking link generation (unique slug per version)
- Track view and download events with IP + user agent
- Analytics aggregation endpoint (totals, unique visitors, timeline)
- Dashboard metrics API: resume count, application funnel, recruiter views

Recruiter Analytics — Deep Dive

This is the platform's most powerful premium differentiator. When a user shares their resume via a tracking link (e.g., [resumeai.ai/r/ahmed-google-fe](#)), every recruiter interaction is silently logged. The user sees a live feed of recruiter activity on their analytics dashboard:

EVENT	DETAIL	TIMESTAMP
■ View	Google · Chrome · macOS	2 days ago
■ View	Google · Chrome · macOS	2 days ago
■ Download	Google · Safari · iPhone	1 day ago
■ View	Google · Firefox · Windows	6 hours ago

Example: 'Google recruiter viewed your resume 3 times. Last seen 6 hours ago.'

Environment Configuration

MongoDB

MONGO_URI

Auth.js

AUTH_SECRET

AUTH_GOOGLE_ID

AUTH_GOOGLE_SECRET

AUTH_GITHUB_ID

AUTH_GITHUB_SECRET

Cloudinary

CLOUDINARY_CLOUD_NAME

CLOUDINARY_API_KEY

CLOUDINARY_API_SECRET

AI

AI_API_KEY

AI_BASE_URL

App

CLIENT_URL

PORT=5000

NODE_ENV=development